

CSS Grid

Master the Two-Dimensional Layout System for Modern Web Design

What is CSS Grid?

CSS Grid is a powerful two-dimensional layout system that allows you to create complex layouts with rows and columns. Unlike Flexbox (one-dimensional), Grid handles both horizontal and vertical spacing, making it ideal for page layouts, dashboards, and multi-dimensional designs.

Grid vs Flexbox

- Flexbox:** One-dimensional, best for rows OR columns (e.g., navigation bars, button groups)
- Grid:** Two-dimensional, perfect for rows AND columns simultaneously (e.g., page layouts, dashboards)
- Best Practice:** Use both together! Grid for overall layout, Flexbox for component alignment.

Grid Container Properties

1. display: grid

Converts an element into a grid container, enabling CSS Grid layout for all direct children (grid items).

```
.container {
  display: grid;
  /* or */ display: inline-grid;
}
```

2. grid-template-columns

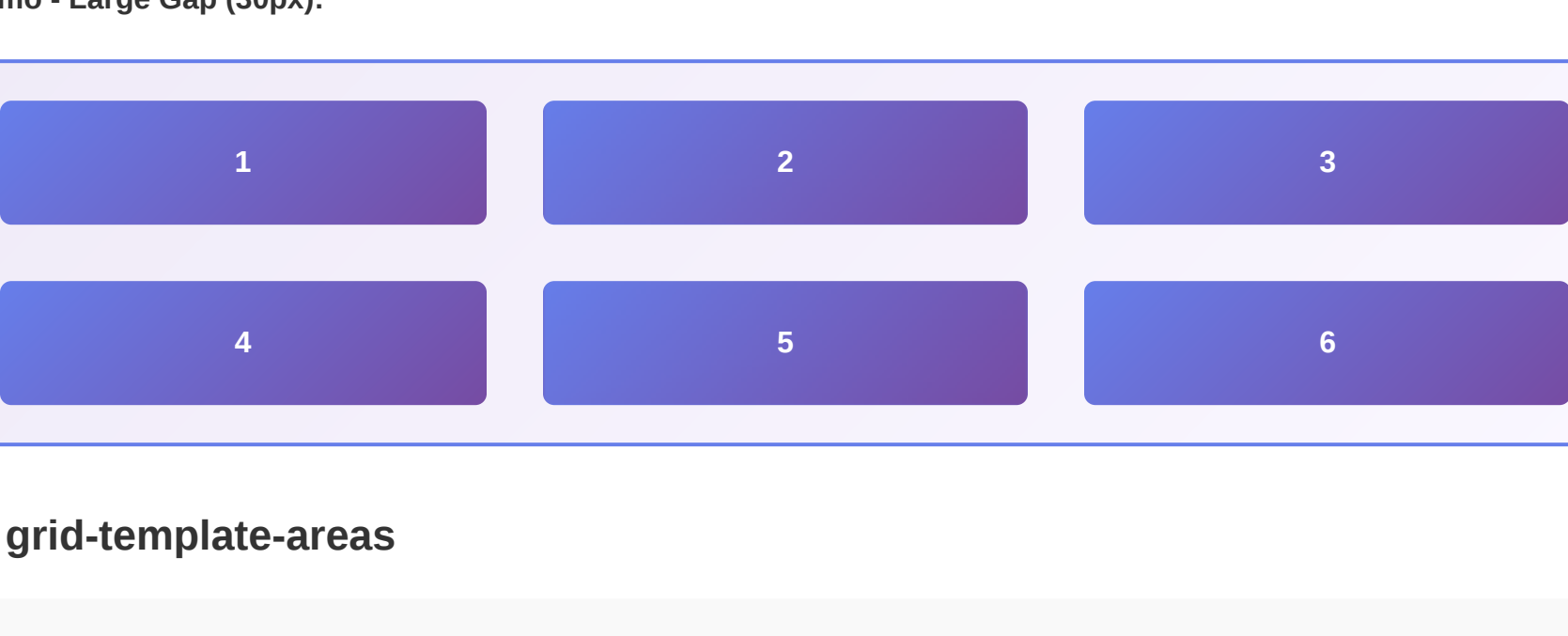
Defines the number and size of columns in the grid. Can use fixed sizes (px), flexible sizes (fr), or percentages.

```
.container {
  grid-template-columns: 200px 1fr 200px;
  /* 3 columns: fixed, flexible, fixed */

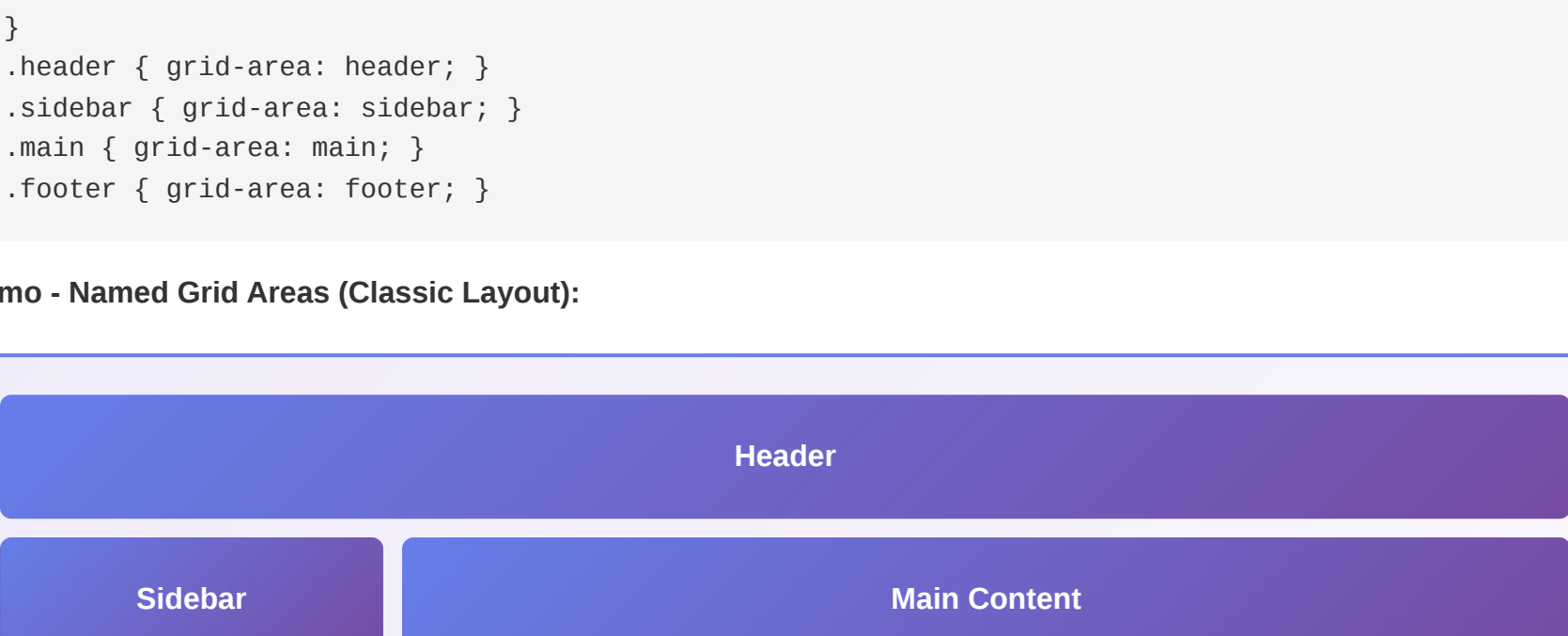
  grid-template-columns: repeat(3, 1fr);
  /* 3 equal columns */

  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  /* Responsive: auto-sized columns */
}
```

Demo - 3 Equal Columns:



Demo - 4 Equal Columns:

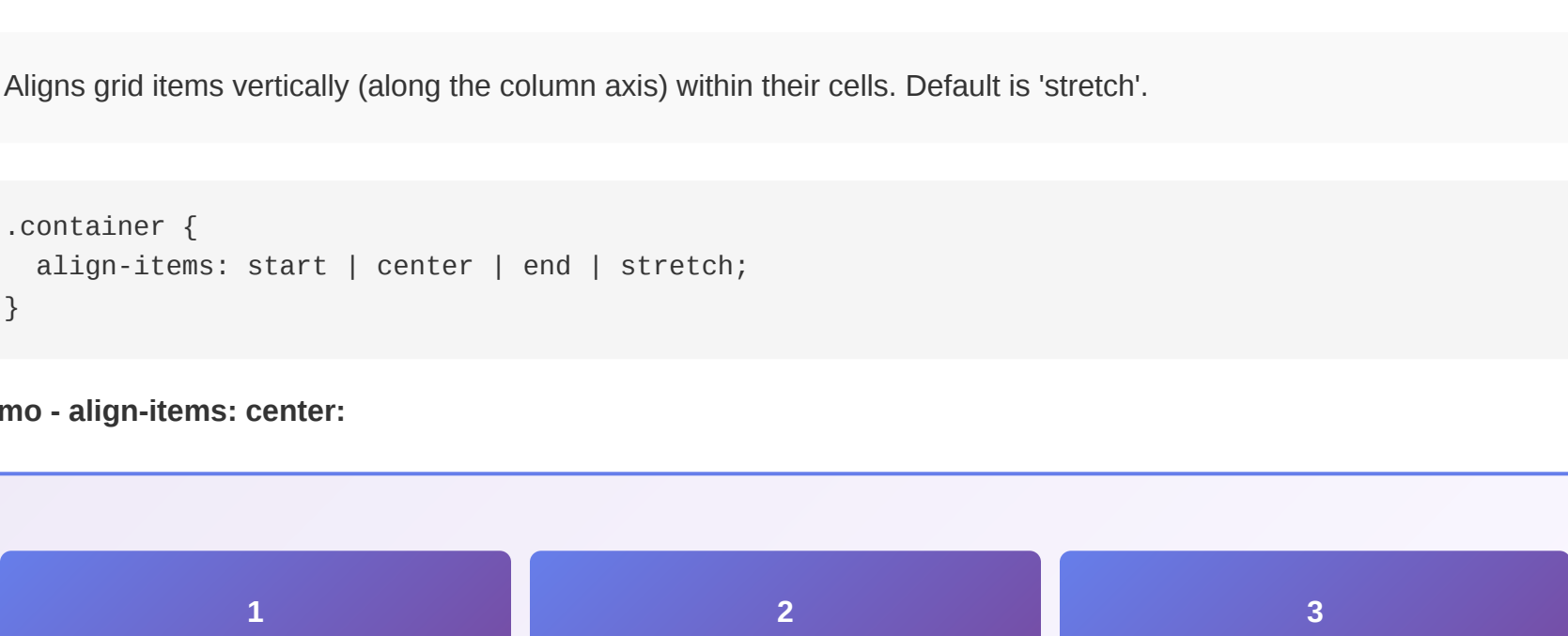


3. grid-template-rows

Defines the number and size of rows in the grid. Follows the same sizing rules as columns.

```
.container {
  grid-template-rows: 100px auto 80px;
  /* 3 rows: fixed, auto, fixed */
}
```

Demo - Grid with Defined Rows:

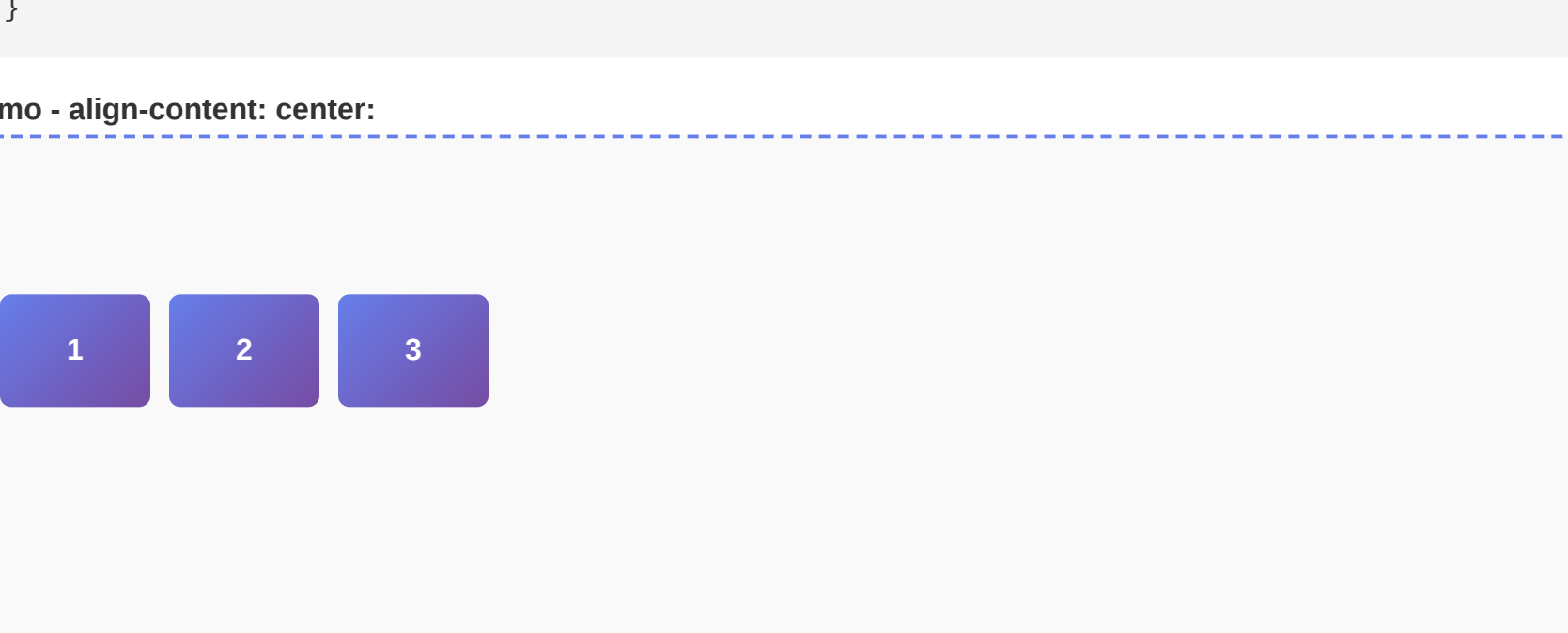


4. gap (row-gap, column-gap)

Sets consistent spacing between grid items. Can be a single value or separate row/column gaps.

```
.container {
  gap: 20px;
  /* or */ row-gap: 20px; column-gap: 15px;
}
```

Demo - Large Gap (30px):

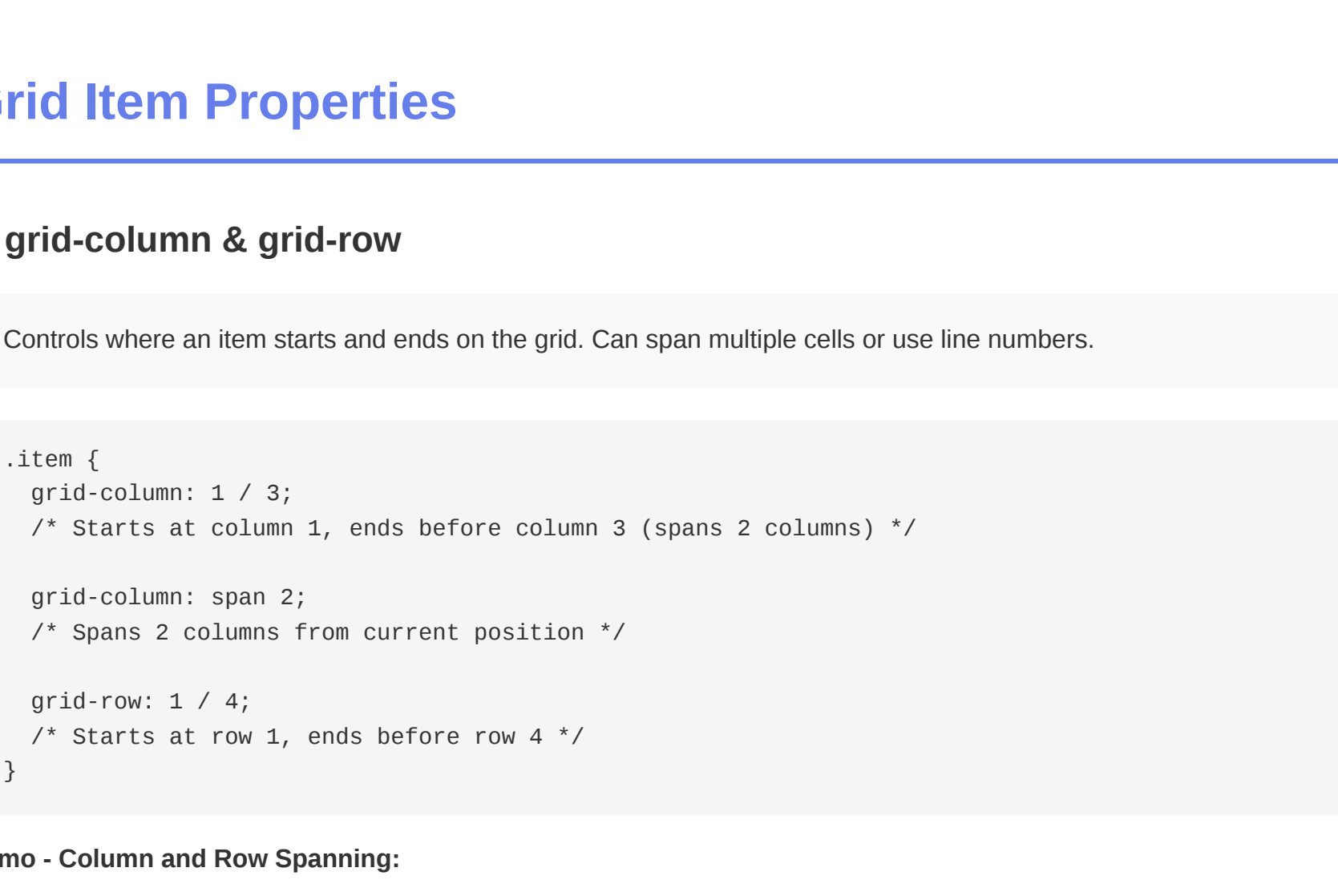


5. grid-template-areas

Defines a visual grid layout using named areas. Makes complex layouts very readable and maintainable.

```
.container {
  display: grid;
  grid-template-columns: 200px 1fr 200px;
  grid-template-areas:
    "header header header"
    "sidebar main main"
    "footer footer footer";
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }
```

Demo - Named Grid Areas (Classic Layout):

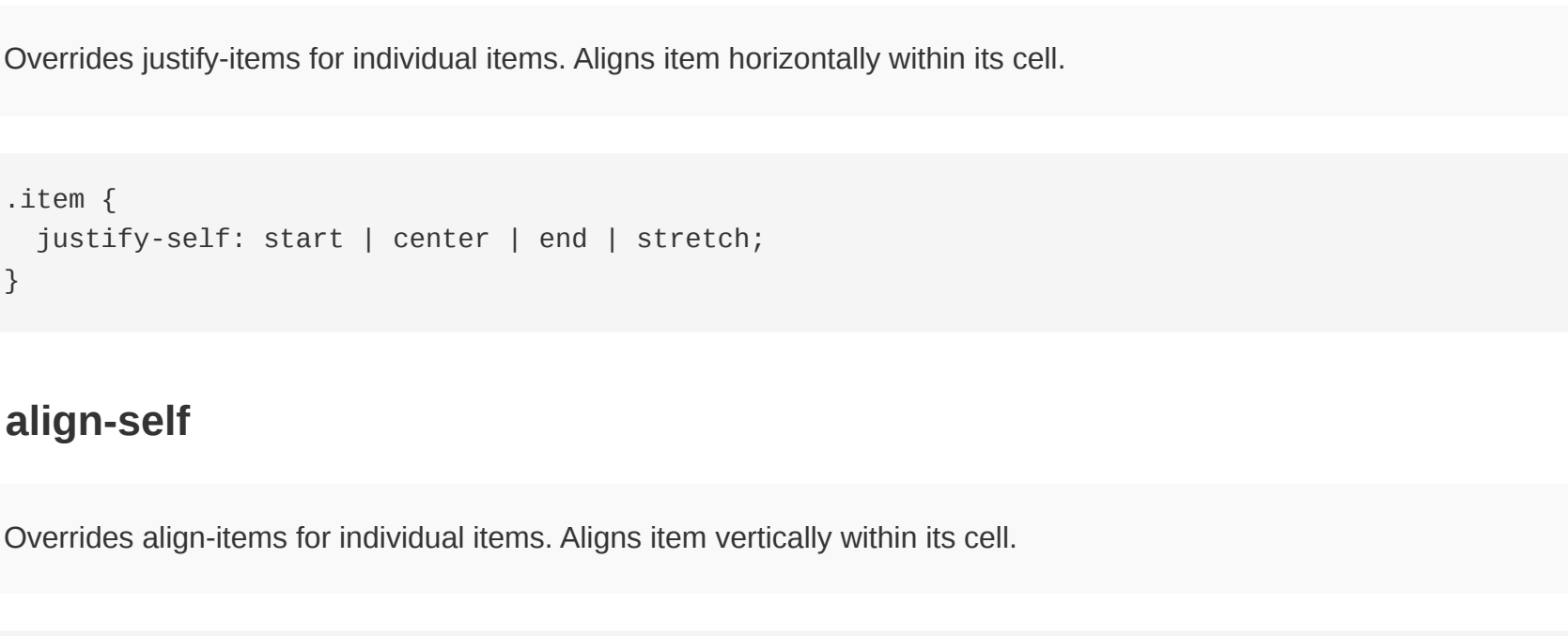


6. justify-items

Aligns grid items horizontally (along the row axis) within their cells.

```
.container {
  justify-items: start | center | end | stretch;
}
```

Demo - justify-items: center:

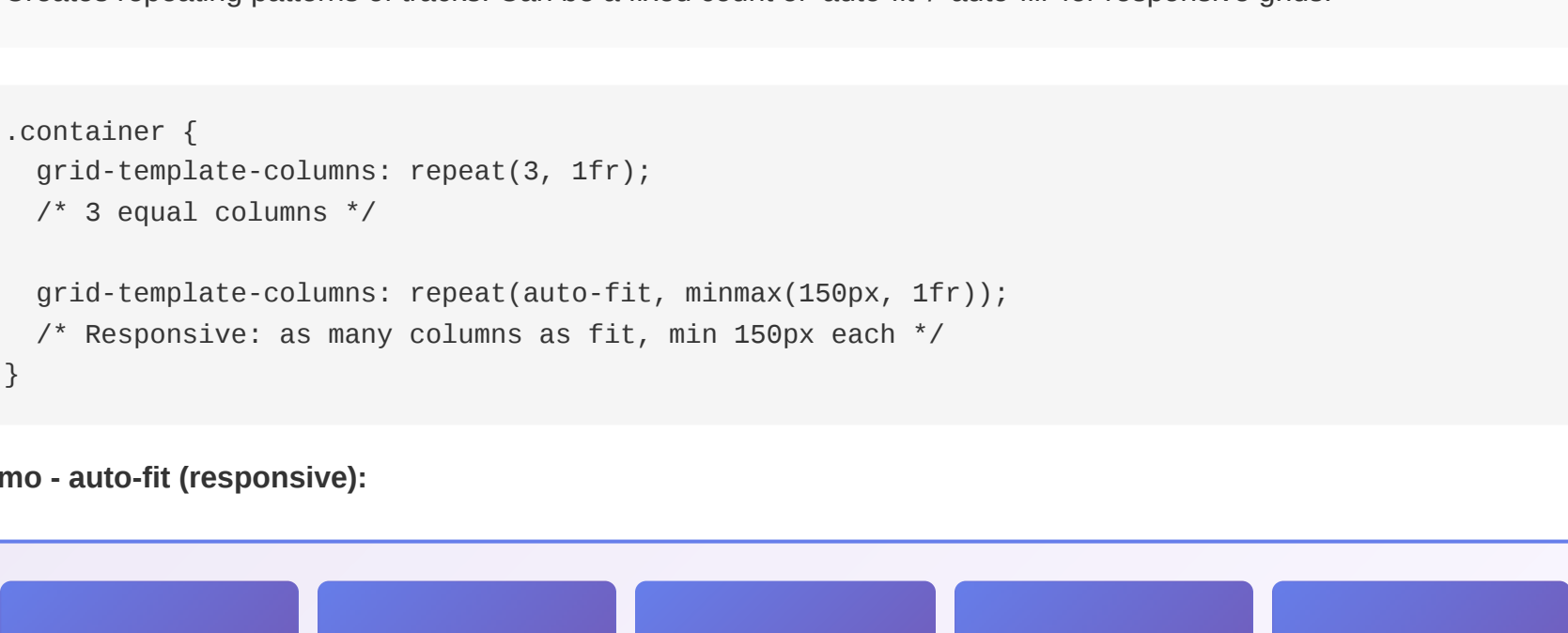


7. align-items

Aligns grid items vertically (along the column axis) within their cells. Default is 'stretch'.

```
.container {
  align-items: start | center | end | stretch;
}
```

Demo - align-items: center:

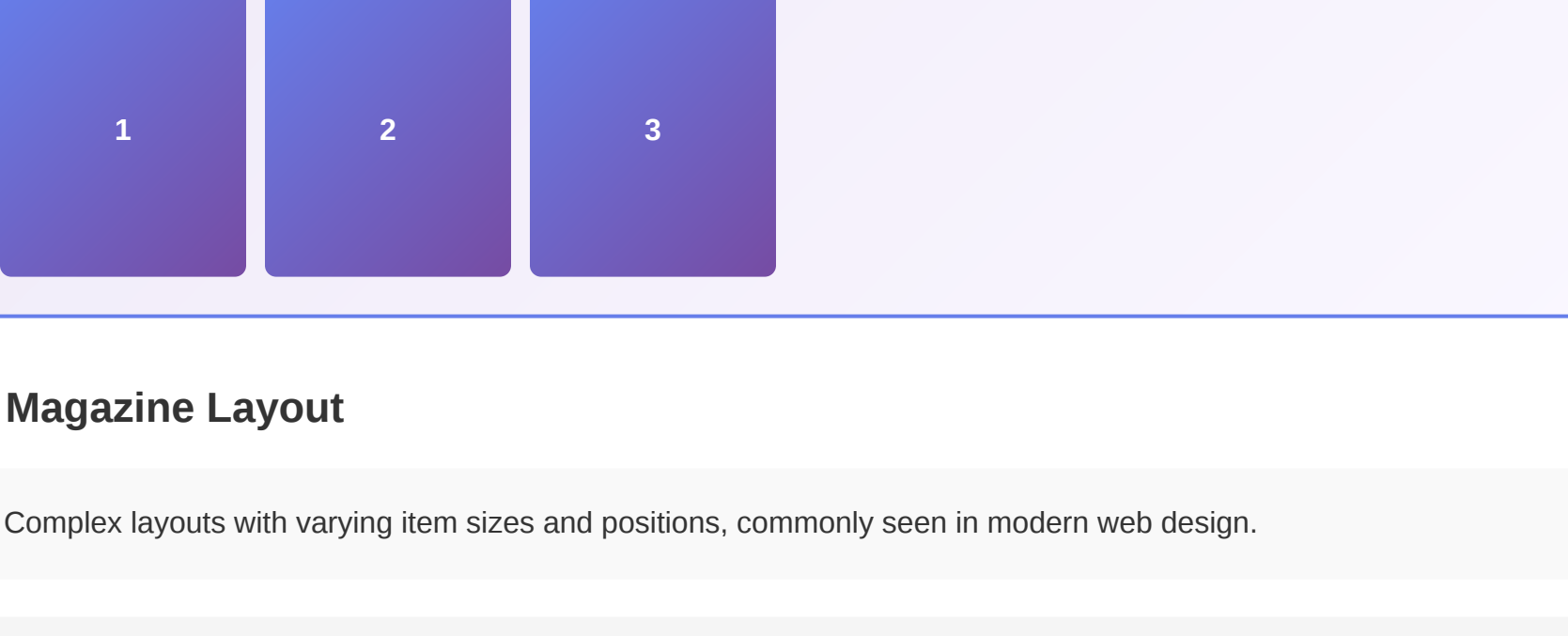


8. justify-content

Distributes grid tracks along the row axis (when grid width is less than container width).

```
.container {
  justify-content: start | center | end | space-between | space-around | space-evenly;
}
```

Demo - justify-content: center:

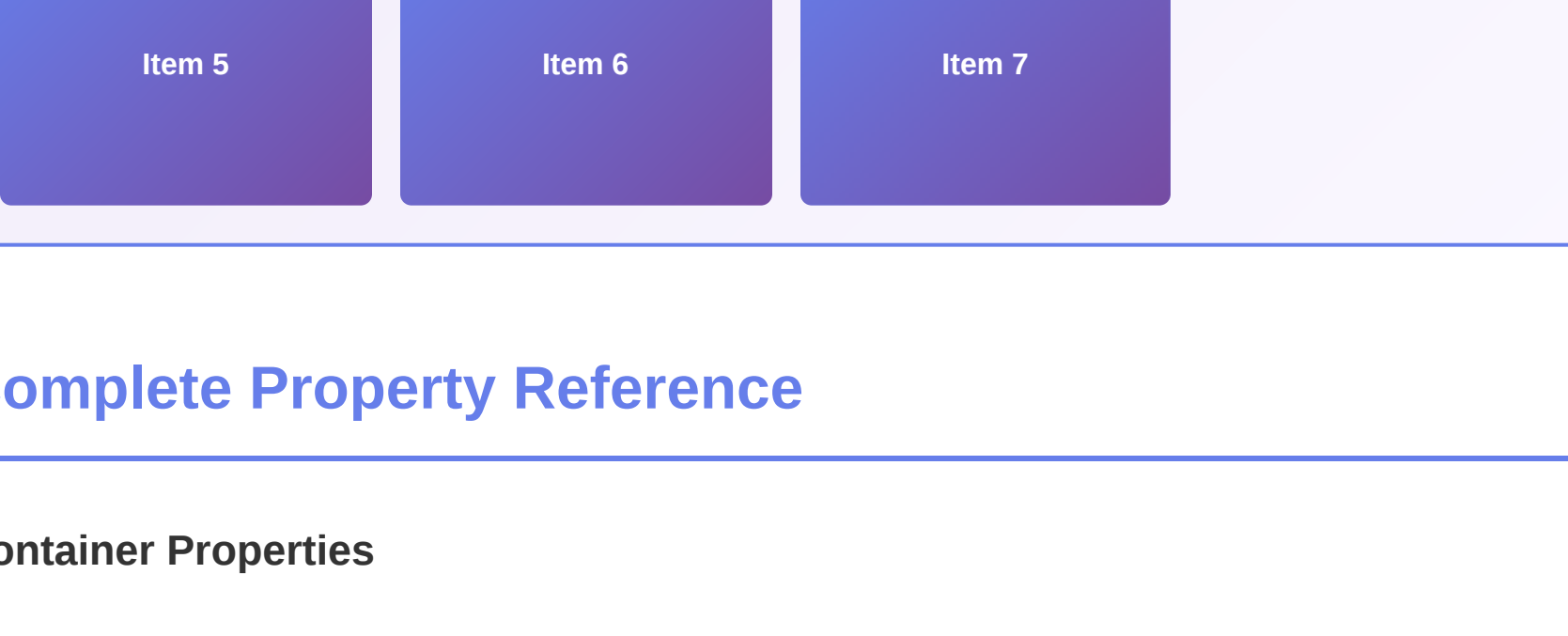


9. align-content

Distributes grid tracks along the column axis (when grid height is less than container height).

```
.container {
  align-content: start | center | end | space-between | space-around | space-evenly;
}
```

Demo - align-content: center:

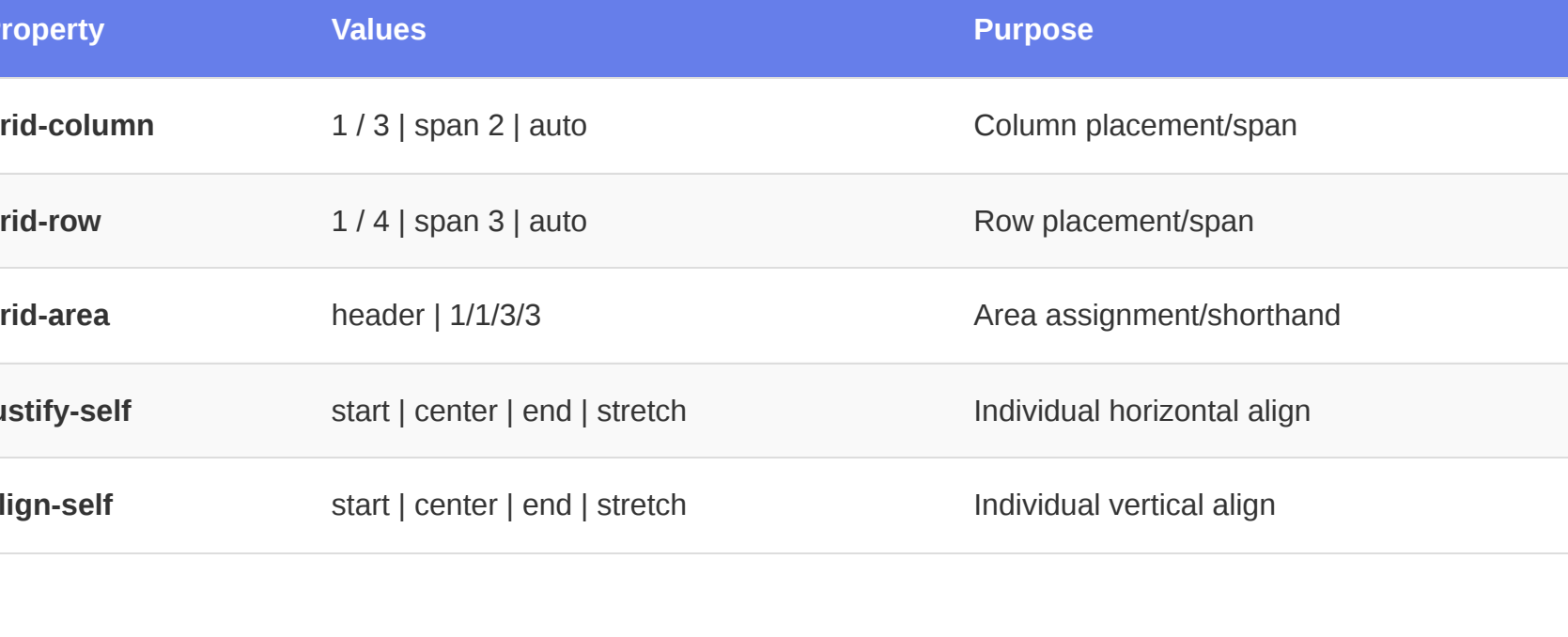


10. grid-auto-flow

Controls how auto-placed items flow into the grid. Default is 'row'.

```
.container {
  grid-auto-flow: row | column | dense;
}
```

Demo - grid-auto-flow: row:



11. grid-auto-rows & grid-auto-columns

Sets the size of automatically generated rows or columns (when items exceed defined tracks).

```
.container {
  grid-auto-rows: 80px;
  /* All auto rows will be 80px tall */
}
```

Grid Item Properties

1. grid-column & grid-row

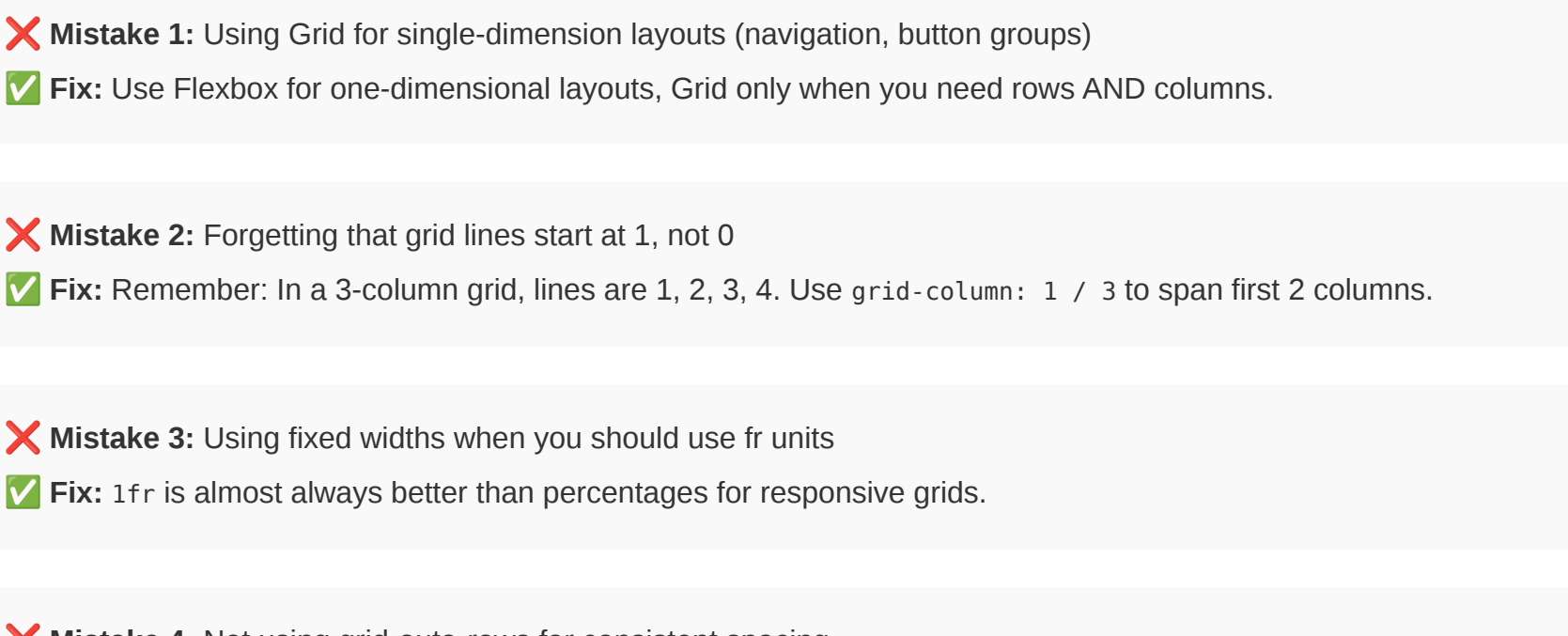
Controls where an item starts and ends on the grid. Can span multiple cells or use line numbers.

```
.item {
  grid-column: 1 / 3;
  /* Starts at column 1, ends before column 3 (spans 2 columns) */

  grid-column: span 2;
  /* Spans 2 columns from current position */

  grid-row: 1 / 4;
  /* Starts at row 1, ends before row 4 */
}
```

Demo - Column and Row Spanning:



2. grid-area

Shorthand for grid-row-start, grid-column-start, grid-row-end, and grid-column-end. Or assign to named grid areas.

```
.item {
  grid-area: header;
  /* Assign to named area */

  grid-area: 1 / 1 / 3 / 3;
  /* row-start / col-start / row-end / col-end */
}
```

3. justify-self

Overrides justify-items for individual items. Aligns item horizontally within its cell.

```
.item {
  justify-self: start | center | end | stretch;
}
```

4. align-self

Overrides align-items for individual items. Aligns item vertically within its cell.

```
.item {
  align-self: start | center | end | stretch;
}
```

Advanced Features

1. The fr Unit (Fractional Unit)

The **fr** unit represents a fraction of available space. It's the most powerful feature of CSS Grid for responsive layouts without media queries.

```
.container {
  grid-template-columns: 1fr 2fr 1fr;
  /* 1:2:1 ratio - middle column is twice as wide */

  grid-template-columns: 200px 1fr 200px;
  /* Fixed + flexible + flexible + fixed */
}
```

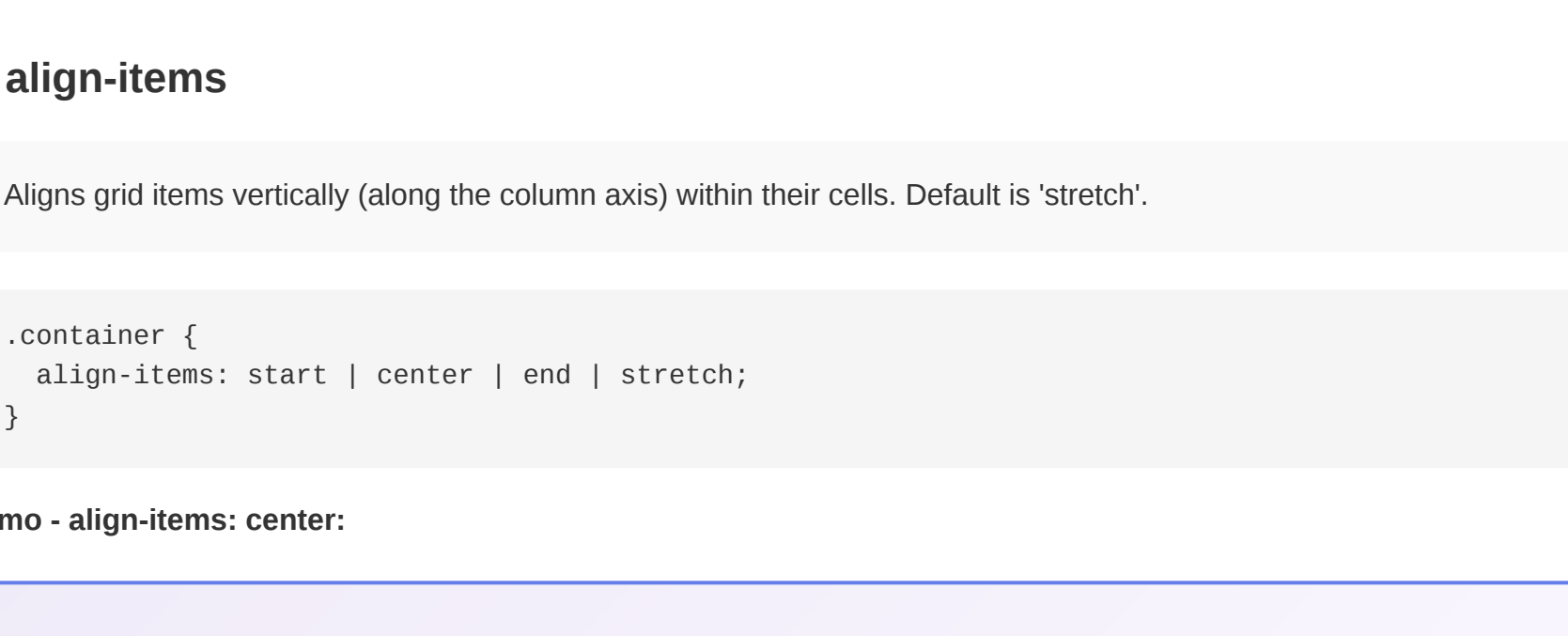
2. repeat() Function

Creates repeating patterns of tracks. Can be a fixed count or 'auto-fit' / 'auto-fill' for responsive grids.

```
.container {
  grid-template-columns: repeat(3, 1fr);
  /* 3 equal columns */

  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  /* Responsive: as many columns as fit, min 150px each */
}
```

Demo - auto-fit (responsive):



3. minmax() Function

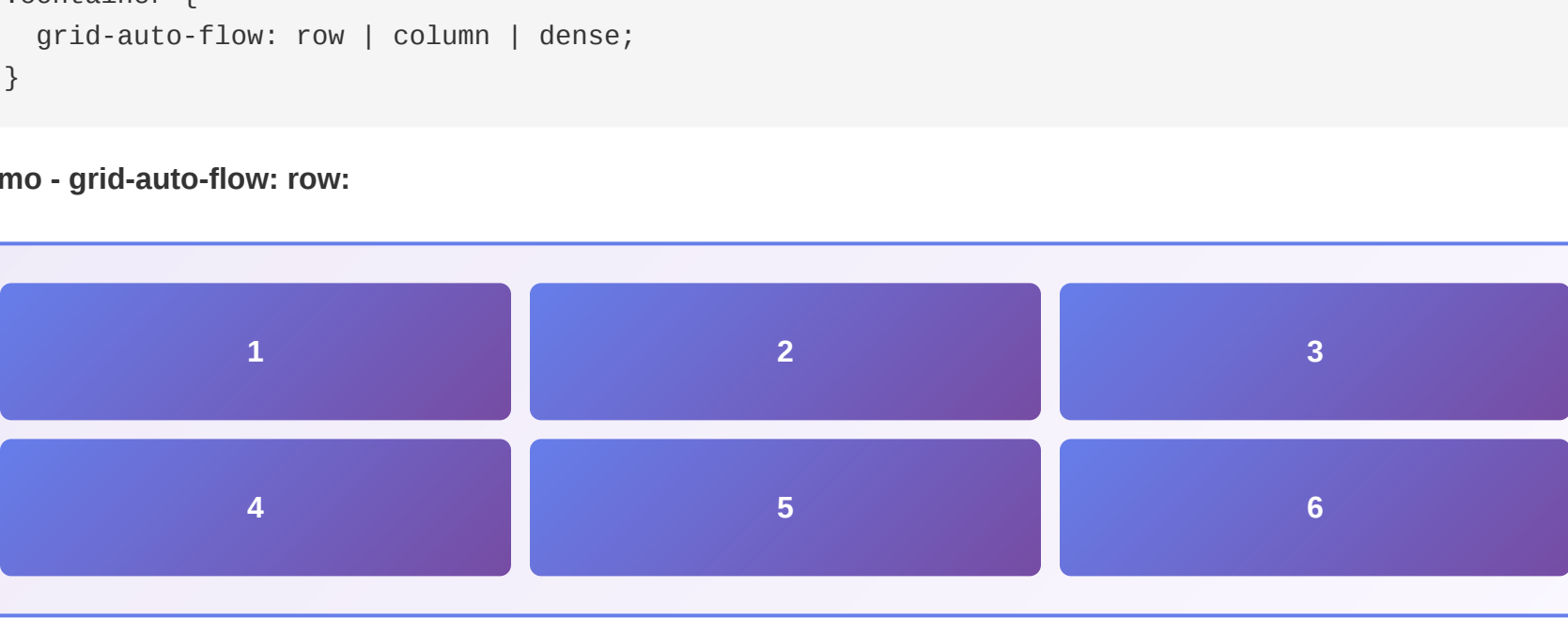
Sets minimum and maximum sizes for grid tracks. Essential for responsive design that adapts to content.

```
.container {
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  /* Columns: minimum 200px, maximum 1fr of available space */
}
```

4. auto-fit vs auto-fill

- auto-fit:** Collapses empty tracks, causing remaining items to expand
- auto-fill:** Keeps empty tracks, items maintain their size

Demo - auto-fill (keeps empty columns):

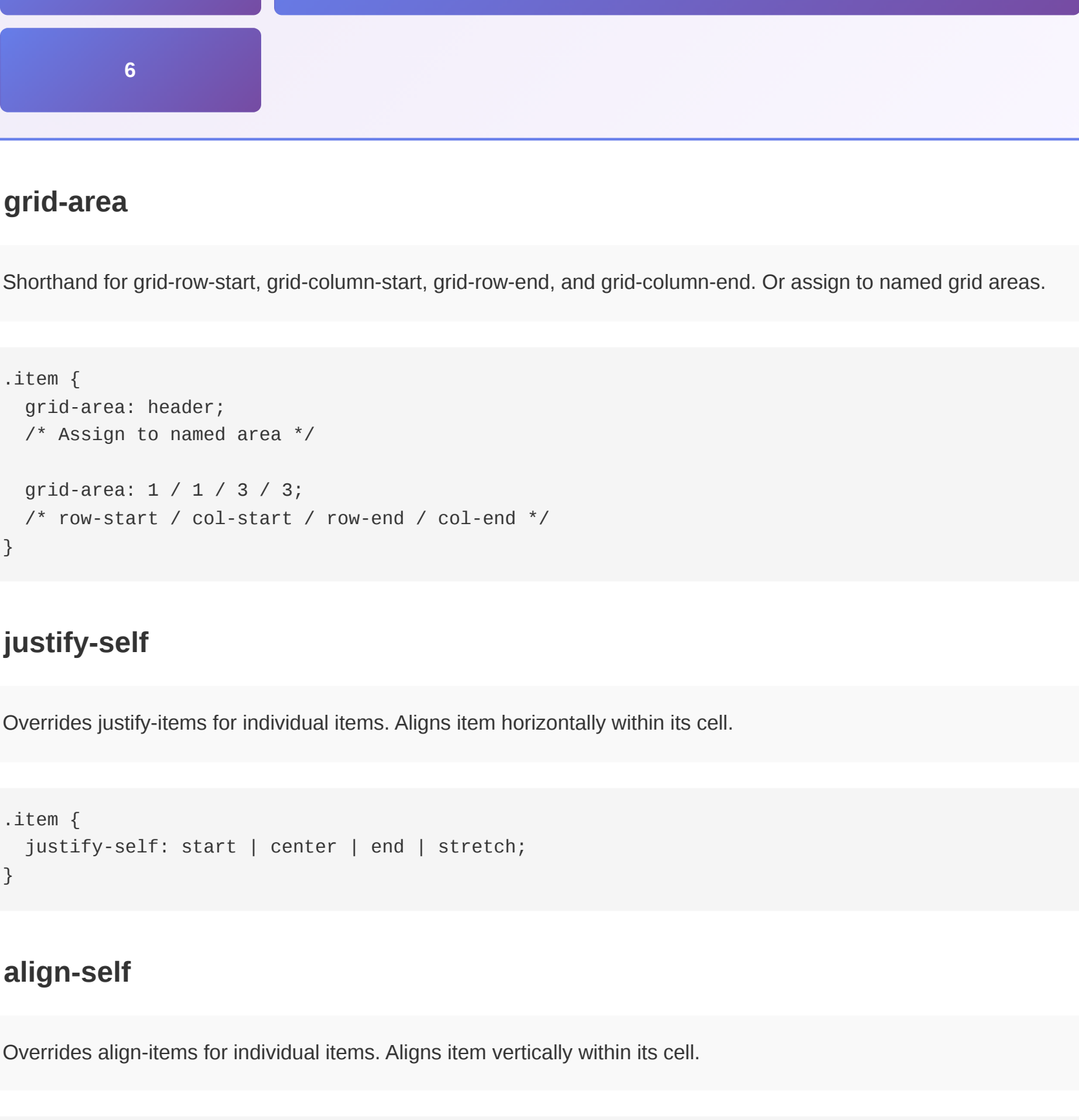


5. Magazine Layout

Complex layouts with varying item sizes and positions, commonly seen in modern web design.

```
.container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: minmax(150px, auto);
  gap: 15px;
}
.featured {
  grid-column: span 2;
  grid-row: span 2;
}
```

Demo - Magazine Layout:



Complete Property Reference

Container Properties

Property	Values	Purpose
display	grid inline-grid	Enables CSS Grid
grid-template-columns	200px 1fr repeat(3, 1fr) auto	Define columns
grid-template-rows	100px auto repeat(2, 1fr)	Define rows
gap	20px 10px 20px	Space between items
grid-template-areas	"header header" "main main"	Named layout areas
justify-items	start center end stretch	Horizontal item alignment
align-items	start center end stretch	Vertical item alignment
justify-content	start center end space-between	Horizontal grid alignment
align-content	start center end space-between	Vertical grid alignment
grid-auto-flow	row column dense	Auto-placement direction

Item Properties

Property	Values	Purpose
grid-column	1 / 3 span 2 auto	Column placement/span
grid-row	1 / 4 span 3 auto	Row placement/span
grid-area	header 1/1/3/3	Area assignment/shorthand
justify-self	start center end stretch	Individual horizontal align
align-self	start center end stretch	Individual vertical align

Best Practices & Tips

- Tip 1: Use fr Units for Responsive Layouts**
The **fr** unit handles responsive sizing without media queries. `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));` is incredibly powerful.
- Tip 2: grid-template-areas for Readability**
Named areas make complex layouts crystal clear and easy to maintain. Much better than line numbers for large projects.
- Tip 3: Combine Grid + Flexbox**
Use Grid for page-level layout, Flexbox for component-level alignment. They work beautifully together!
- Tip 4: minmax() for Flexible Constraints**
`minmax(200px, 1fr)` ensures columns never collapse below 200px while growing as needed.
- Tip 5: Grid Lines Get Named Numbers**
In a 3-column grid, you have 4 lines: 1, 2, 3, 4 (from left to right). Negative numbers work backwards (-1 is rightmost).
- Tip 6: dense for Optimal Packing**
`grid-auto-flow: dense` fills grid gaps intelligently, useful for masonry-like layouts.

Common Layout Patterns

- Pattern 1: Sidebar Layout**
`grid-template-columns: 250px 1fr;` - Fixed sidebar, flexible content
- Pattern 2: Holy Grail**
`grid-template-columns: 1fr; grid-template-areas: "header" "main" "footer";` - Full-width responsive
- Pattern 3: Dashboard Grid**
`grid-template-columns: repeat(auto-fit, minmax(300px, 1fr)); gap: 20px;` - Card-based layout
- Pattern 4: Asymmetric Layout**
`grid-template-columns: 2fr 1fr 1fr;` - Different column widths

Browser Support

- ✅ **Excellent Support Across All Modern Browsers**
Chrome 57+, Firefox 52+, Safari 10.1+, Edge 16+
- ⚠️ **Limited support:** IE 11 (partial/legacy grid spec)
- 🔧 **Recommendation:** CSS Grid is safe to use in all modern projects. IE 11 support requires fallbacks.

Common Mistakes to Avoid

- ❌ **Mistake 1:** Using Grid for single-dimension layouts (navigation, button groups)
- ✅ **Fix:** Use Flexbox for one-dimensional layouts. Grid only when you need rows AND columns.
- ❌ **Mistake 2:** Forgetting that grid lines start at 1, not 0
- ✅ **Fix:** Remember: In a 3-column grid, lines are 1, 2, 3, 4. Use `grid-column: 1 / 3` to span first 2 columns.
- ❌ **Mistake 3:** Using fixed widths when you should use **fr** units
- ✅ **Fix:** `1fr` is almost always better than percentages for responsive grids.
- ❌ **Mistake 4:** Not using `grid-auto-rows` for consistent spacing
- ✅ **Fix:** Set `grid-auto-rows: minmax(150px, auto)` to prevent inconsistent item heights.
- ❌ **Mistake 5:** Applying grid properties to items (grid items can't have `grid-template-*`)
- ✅ **Fix:** Grid container gets `grid-template-*` properties, items get `grid-column`, `grid-row`, `grid-area`.